

ARIMA Model Testing

```
In [1]: import itertools
import logging
import os
import time
import warnings
from concurrent.futures import ThreadPoolExecutor

import matplotlib.pyplot as plt
import mlflow
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from dotenv import load_dotenv
from joblib import Parallel, delayed, parallel_config
from scipy import stats
from sklearn.exceptions import UndefinedMetricWarning
from sklearn.metrics import r2_score
from statsmodels.tsa.statespace.sarimax import SARIMAX
from tqdm import tqdm

load_dotenv()

ROOT_PATH = os.path.dirname(os.getcwd())
DATA_FOLDER_PATH = os.path.join(ROOT_PATH, 'data', 'processed')

COVID_START = pd.Timestamp('2020-01-01')
COVID_END = pd.Timestamp('2021-12-31')
POST_COVID_START = pd.Timestamp('2022-01-01')
COVID_RECOVERY_END = pd.Timestamp('2022-06-30')

sns.set_palette('colorblind')
logging.getLogger('mlflow').setLevel(logging.ERROR)
warnings.filterwarnings('ignore', category=UndefinedMetricWarning)
warnings.filterwarnings('ignore', category=FutureWarning, module='mlflow')
mlflow.set_tracking_uri(os.getenv('MLFLOW_TRACKING_URI'))
EXPERIMENT_NAME = 'Brazil Tourism Forecasting ARIMA'
experiment = mlflow.set_experiment(EXPERIMENT_NAME)
```

Target Transformations

Three target transformations are compared in the grid. The model is always evaluated on the **original scale**: forecasts are inverse-transformed back to arrival counts before computing the validation metrics.

Option	Transform	Inverse
none	identity (raw arrival counts)	identity

Option	Transform	Inverse
log	log(y)	exp($\hat{y}$)
auto_stationary	AutoStationaryTransformer (AutoML)	exact inverse_transform of the fitted pipeline

auto_stationary comes from the companion repository of:

Joseph, M. & Tackes, J. *Modern Time Series Forecasting with Python*, 2nd ed. (Packt, 2024) — [PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E](#), MIT License

During `fit` (`src/transforms/target_transformations.py`) it runs the tests explored in notebook 3 and adds a step only when the test asks for it:

1. `check_trend` (Mann-Kendall). If a trend is found → `DetrendingTransformer` (linear).
2. `check_seasonality` at `m = 12`. If seasonal → `DeseasonalizingTransformer` (period averages).
3. `check_heteroscedasticity` (White). If heteroscedastic → `AddMTTransformer` when needed, then `BoxCoxTransformer` with Guerrero λ .

The transformer is fitted separately on each training window.

The repository is expected as a sibling folder of this project (override with the `MTSF_REPO_PATH` environment variable).

```
In [2]: import sys

# Helpers credited to: Joseph, M. & Tackes, J. "Modern Time Series Forecasting with
# Python", 2nd ed. (Packt, 2024) – MIT License
# github.com/PacktPublishing/Modern-Time-Series-Forecasting-with-Python-2E
MTSF_REPO_PATH = os.getenv(
    'MTSF_REPO_PATH',
    os.path.join(
        os.path.dirname(ROOT_PATH), 'Modern-Time-Series-Forecasting-with-Python-2E'
    ),
)
if MTSF_REPO_PATH not in sys.path:
    sys.path.insert(0, MTSF_REPO_PATH)

from src.transforms.target_transformations import ( # noqa: E402
    AutoStationaryTransformer,
    BoxCoxTransformer,
)
from src.utils.ts_utils import forecast_bias, mase, metrics_adapter # noqa: E402

SEASONAL_PERIOD = 12
FREQ = 'ME'
```

```

def fit_auto_stationary(train_series):
    """Fit a fresh AutoStationaryTransformer (Mann-Kendall trend, Guerrero Box-Cox)
    # fresh dicts every fit: the transformer mutates its parameter dicts in place
    transformer = AutoStationaryTransformer(
        confidence=0.05,
        seasonal_period=SEASONAL_PERIOD,
        trend_check_params={'mann_kendall': True},
        detrender_params={'degree': 1},
        deseasonalizer_params={},
        box_cox_params={'optimization': 'guerrero'},
    )
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        return transformer.fit(train_series, freq=FREQ)

def describe_pipeline(transformer):
    steps = [
        type(tr).__name__.replace('Transformer', '') for tr in transformer._pipeline
    ]
    return ' → '.join(steps) if steps else '(none)'

class IdentityTarget:
    """No target transformation."""

    @staticmethod
    def transform(y):
        return y

    @staticmethod
    def inverse_transform(y):
        return y

class LogTarget:
    """Natural log of the arrival counts; inverse is exp."""

    @staticmethod
    def transform(y):
        return np.log(y)

    @staticmethod
    def inverse_transform(y):
        return np.exp(y)

def fit_target_transform(name, train_series):
    """Return a fitted transformer exposing `transform` / `inverse_transform`."""
    if name == 'none':
        return IdentityTarget()
    if name == 'log':
        return LogTarget()
    if name == 'auto_stationary':
        return fit_auto_stationary(train_series)
    raise ValueError(f'unknown target transform: {name}')

```

Data Preparation

Several training data selections are evaluated in the grid search:

- `last_{14,12,10,8,6,4}y_covid_exog` — last N years with a 0/1 COVID regressor (`is_covid`) to absorb the structural break
- `last_{14,12,10,8,6,4}y_covid_recovery_exog` — same windows but with **two** regressors: `is_covid` (Jan 2020–Dec 2021) **and** `is_covid_recovery` (Jan 2022–Jun 2022), the 12-month rebound period immediately after the pandemic
- `post_covid` — only post-COVID data (2022 onwards); no exog needed, cleaner signal
- `post_covid_recovery_exog` — same post-COVID window with `is_covid_recovery` (Jan 2022–Jun 2022) to explicitly model the rebound year within that short series

Every selection is crossed with the three target transformations (`none`, `log`, `auto_stationary`). The `AutoStationaryTransformer` is fitted on each training window only (the raw arrival counts).

Evaluation metrics are computed on **val.parquet** (the validation year: the 12 months before the test year, split in notebook 2) for all models, after inverse-transforming the forecasts back to arrival counts. The model is **selected** on the validation year; the test set is never read here and is kept for the final notebook, which refits the selected model and evaluates it once. The training windows are counted back from the last training month, so every date in this notebook follows the data.

```
In [3]: train_df = pd.read_parquet(os.path.join(DATA_FOLDER_PATH, 'train.parquet'))
train_monthly = train_df.set_index('date')[['arrival_count']].resample('ME').sum()
train_monthly['is_covid'] = (
    (train_monthly.index >= COVID_START) & (train_monthly.index <= COVID_END)
).astype(float)
train_monthly['is_covid_recovery'] = (
    (train_monthly.index >= POST_COVID_START)
    & (train_monthly.index <= COVID_RECOVERY_END)
).astype(float)

val_df = pd.read_parquet(os.path.join(DATA_FOLDER_PATH, 'val.parquet'))
val_monthly = val_df.set_index('date')[['arrival_count']].resample('ME').sum()
val_monthly['is_covid'] = 0.0
val_monthly['is_covid_recovery'] = 0.0
assert val_monthly.index[0] == train_monthly.index[-1] + pd.offsets.MonthEnd(1)

window_years = [14, 12, 10, 8, 6, 4]
train_windows = {}
for years in window_years:
    start = train_monthly.index[-1] - pd.DateOffset(years=years)
    train_windows[years] = train_monthly[train_monthly.index >= start].copy()

train_post_covid = train_monthly[train_monthly.index >= POST_COVID_START].copy()
```

```

print(
    f'Full train : {train_monthly.index[0].strftime("%b %Y")} → {train_monthly.index[-1].strftime("%b %Y")} (428 obs)'
)
for years in window_years:
    tw = train_windows[years]
    print(
        f'Last {years}>2y : {tw.index[0].strftime("%b %Y")} → {tw.index[-1].strftime("%b %Y")} ({tw.index[-1].index[0].year - tw.index[0].year + 1} obs)'
    )
print(
    f'Post-COVID : {train_post_covid.index[0].strftime("%b %Y")} → {train_post_covid.index[-1].strftime("%b %Y")} (32 obs)'
)
print(
    f'Validation : {val_monthly.index[0].strftime("%b %Y")} → {val_monthly.index[-1].strftime("%b %Y")} (12 obs)'
)

```

```

Full train : Jan 1989 → Aug 2024 (428 obs)
Last 14y : Aug 2010 → Aug 2024 (169 obs, COVID obs: 24, recovery obs: 6)
Last 12y : Aug 2012 → Aug 2024 (145 obs, COVID obs: 24, recovery obs: 6)
Last 10y : Aug 2014 → Aug 2024 (121 obs, COVID obs: 24, recovery obs: 6)
Last 8y : Aug 2016 → Aug 2024 (97 obs, COVID obs: 24, recovery obs: 6)
Last 6y : Aug 2018 → Aug 2024 (73 obs, COVID obs: 24, recovery obs: 6)
Last 4y : Aug 2020 → Aug 2024 (49 obs, COVID obs: 17, recovery obs: 6)
Post-COVID : Jan 2022 → Aug 2024 (32 obs)
Validation : Sep 2024 → Aug 2025 (12 obs)

```

Evaluation Metrics and Baselines

Every model is scored on the validation year, on the original scale (arrival counts), with the same six metrics as notebook 7:

Metric	Definition	Reading	--- --- ---	RMSE	$\sqrt{\text{mean}((y - \hat{y})^2)}$	Ranking metric of the grid; penalizes large misses
MAE	$\text{mean}(y - \hat{y})$	Typical monthly miss, in arrivals				
MAPE	$\text{mean}(y - \hat{y} / y) \times 100$	Typical miss in %				
R²	$1 - \text{SSE} / \text{SST}$	Share of the validation-year variation explained				
MASE	$\text{MAE} / \text{MAE}(\text{one-step naive on the training series})$	Below 1: the model beats a naive forecast's in-sample error				
Forecast Bias	$(\sum y - \sum \hat{y}) / \sum y \times 100$	Positive = the model under -forecasts the year				

MASE and Forecast Bias use the book's own functions (`mase`, `forecast_bias` and `metrics_adapter` from `src/utils/ts_utils.py`, the functions behind `calculate_metrics` in notebook 7), so both notebooks compute them identically. The MASE scale is the **full** training series, so it is comparable across training windows.

Baselines (the references of chapter 8, as in notebook 7):

- **naive**: every validation month repeats the last training month.
- **seasonal naive**: every validation month repeats the same month one year earlier.

A SARIMAX that does not beat the seasonal naive has not learned anything beyond "same as last year".

```

In [4]: METRICS = ['rmse', 'mae', 'mape', 'r2', 'mase', 'forecast_bias']
METRIC_LABELS = {
    'rmse': 'RMSE',
    'mae': 'MAE',
    'mape': 'MAPE (%)',
    'r2': 'R²',
    'mase': 'MASE',
    'forecast_bias': 'Forecast Bias (%)',
}

val_actual = val_monthly['arrival_count']
mase_insample = train_monthly['arrival_count'] # full training series (MASE scale)

def score(actual, forecast):
    """Validation metrics on the original scale (arrival counts)."""
    forecast = pd.Series(np.asarray(forecast, dtype=float), index=actual.index)
    errors = actual - forecast
    return {
        'rmse': float(np.sqrt(np.mean(errors**2))),
        'mae': float(np.mean(np.abs(errors))),
        'mape': float(np.mean(np.abs(errors / actual)) * 100),
        'r2': float(r2_score(actual, forecast)),
        'mase': float(metrics_adapter(mase, actual, forecast, insample=mase_insample)),
        'forecast_bias': float(metrics_adapter(forecast_bias, actual, forecast)),
    }

baseline_forecasts = {
    'naive': pd.Series(
        train_monthly['arrival_count'].iloc[-1], index=val_monthly.index
    ),
    'seasonal_naive': pd.Series(
        train_monthly['arrival_count']
        .reindex(val_monthly.index - pd.offsets.MonthEnd(12))
        .to_numpy(),
        index=val_monthly.index,
    ),
}
baselines = pd.DataFrame({
    name: score(val_actual, forecast) for name, forecast in baseline_forecasts.items()
}).T
BASELINE_RMSE = float(baselines.loc['seasonal_naive', 'rmse'])
baselines[METRICS].round(3)

```

```

Out[4]:

```

	rmse	mae	mape	r2	mase	forecast_bias
naive	460440.495	319530.000	33.968	-0.929	3.857	43.328
seasonal_naive	255353.116	209126.083	26.478	0.407	2.525	28.357

Model Selection Grid

Grid dimensions:

- **SARIMAX orders:** `p, q ∈ {0,1,2}`, `d ∈ {0,1}`, `P, Q ∈ {0,1}`, `D ∈ {0,1}`, `s = 12` → 144 configs. The differencing orders are **searched**, not fixed, and independently of the target transformation. This way, the grid shows directly whether a transformation (e.g. the AutoML detrend/deseasonalize) replaces differencing or still needs it.
- **Target transformation:** `none`, `log`, `auto_stationary` → 3 options
- **Training data selection:** `last_14y/12y/10y/8y/6y/4y_covid_exog`, `last_14y/12y/10y/8y/6y/4y_covid_recovery_exog`, `post_covid`, or `post_covid_recovery_exog` → 14 options
- **Total runs:** 6,048 — one MLflow run per (order, target transformation, data selection)

Execution. The grid runs in two phases:

1. **Fit in parallel.** Each task (data selection, transformation, order) is fitted and scored in a separate process with `joblib` (`N_JOBS` workers, one BLAS thread each to avoid oversubscription). Workers never touch MLflow; every task catches its own errors and returns them as a failed result.
2. **Log from the main process.** Every result becomes one MLflow run (params + metrics, including `fit_seconds`, in a single `log_batch` call). Runs are logged from `N_LOG_THREADS` threads, because the cost is the HTTP round-trips to the server, not CPU. Failed fits are logged with status `FAILED` and an `error` param.

AIC/BIC are computed on the transformed scale and depend on the differencing, so they are only comparable *within* the same data selection, transformation and `(d, D)`. Models are ranked across the grid by validation RMSE on the original scale.

```
In [5]: s = 12

p_range = range(0, 3)
d_range = range(0, 2)
q_range = range(0, 3)
P_range = range(0, 2)
D_range = range(0, 2)
Q_range = range(0, 2)

sarimax_grid = list(
    itertools.product(p_range, d_range, q_range, P_range, D_range, Q_range)
)

TARGET_TRANSFORMS = ['none', 'log', 'auto_stationary']

# — training data selections (raw target + exogenous regressors) —————
data_selections = {}

for years in window_years:
```

```

tw = train_windows[years]
# COVID indicator only
data_selections[f'last_{years}y_covid_exog'] = {
    'raw_series': tw['arrival_count'],
    'exog_train': tw[['is_covid']],
    'exog_val': val_monthly[['is_covid']],
}
# COVID indicator + recovery indicator (Jan-Jun 2022)
data_selections[f'last_{years}y_covid_recovery_exog'] = {
    'raw_series': tw['arrival_count'],
    'exog_train': tw[['is_covid', 'is_covid_recovery']],
    'exog_val': val_monthly[['is_covid', 'is_covid_recovery']],
}

data_selections['post_covid'] = {
    'raw_series': train_post_covid['arrival_count'],
    'exog_train': None,
    'exog_val': None,
}

# Post-COVID window with recovery flag to model the Jan-Jun 2022 rebound explicitly
data_selections['post_covid_recovery_exog'] = {
    'raw_series': train_post_covid['arrival_count'],
    'exog_train': train_post_covid[['is_covid_recovery']],
    'exog_val': val_monthly[['is_covid_recovery']],
}

# — (data selection × target transformation) configs —————
def make_config(selection, transform_name):
    """Fit the target transformation on the training window and store the pieces."""
    transformer = fit_target_transform(transform_name, selection['raw_series'])
    is_auto = transform_name == 'auto_stationary'
    box_cox = (
        [tr for tr in transformer._pipeline if isinstance(tr, BoxCoxTransformer)]
        if is_auto
        else []
    )
    return {
        **selection,
        'target_transform': transform_name,
        'series': transformer.transform(selection['raw_series']),
        'transformer': transformer,
        'pipeline': describe_pipeline(transformer) if is_auto else transform_name,
        'boxcox_lambda': box_cox[0].boxcox_lambda if box_cox else None,
    }

model_configs = {
    (data_key, transform_name): make_config(selection, transform_name)
    for data_key, selection in data_selections.items()
    for transform_name in TARGET_TRANSFORMS
}

# — what the AutoML pipeline chose for each training window —————
transform_summary = pd.DataFrame([

```

```

{
    'data_selection': data_key,
    'n_train': len(cfg['raw_series']),
    'pipeline': cfg['pipeline'],
    'trend (MK p)': round(cfg['transformer']._trend_check['p_value'], 4),
    'seasonal': cfg['transformer']._seasonality_check['seasonal'],
    'hetero (White p)': round(cfg['transformer']._hetero_check['lm_p_value'], 4),
    'boxcox_λ': None
    if cfg['boxcox_lambda'] is None
    else round(cfg['boxcox_lambda'], 3),
}
for (data_key, transform_name), cfg in model_configs.items()
    if transform_name == 'auto_stationary'
]).set_index('data_selection')

print(f'SARIMAX configs      : {len(sarimax_grid)}')
print(f'Target transformations: {len(TARGET_TRANSFORMS)}')
print(f'Data selections      : {len(data_selections)}')
print(f'Total runs           : {len(sarimax_grid) * len(model_configs)}')
transform_summary

```

```

SARIMAX configs      : 144
Target transformations: 3
Data selections      : 14
Total runs           : 6048

```

Out[5]:

	n_train	pipeline	trend (MK p)	seasonal	hetero (White p)	boxcox_λ
data_selection						
last_14y_covid_exog	169	Detrending → Deseasonalizing → AddM → BoxCox	0.0061	True	0.0048	1.649
last_14y_covid_recovery_exog	169	Detrending → Deseasonalizing → AddM → BoxCox	0.0061	True	0.0048	1.649
last_12y_covid_exog	145	Detrending → Deseasonalizing → AddM → BoxCox	0.0017	True	0.0397	2.000
last_12y_covid_recovery_exog	145	Detrending → Deseasonalizing → AddM → BoxCox	0.0017	True	0.0397	2.000
last_10y_covid_exog	121	Detrending → Deseasonalizing → AddM → BoxCox	0.0041	True	0.0015	2.000
last_10y_covid_recovery_exog	121	Detrending → Deseasonalizing → AddM → BoxCox	0.0041	True	0.0015	2.000
last_8y_covid_exog	97	Deseasonalizing → AddM → BoxCox	0.0741	True	0.0045	2.000
last_8y_covid_recovery_exog	97	Deseasonalizing → AddM → BoxCox	0.0741	True	0.0045	2.000
last_6y_covid_exog	73	Deseasonalizing → AddM → BoxCox	0.1658	True	0.0152	1.688
last_6y_covid_recovery_exog	73	Deseasonalizing → AddM → BoxCox	0.1658	True	0.0152	1.688
last_4y_covid_exog	49	Deseasonalizing	0.0583	True	0.4822	NaN
last_4y_covid_recovery_exog	49	Deseasonalizing	0.0583	True	0.4822	NaN
post_covid	32	Deseasonalizing	0.2085	True	0.0900	NaN
post_covid_recovery_exog	32	Deseasonalizing	0.2085	True	0.0900	NaN

MLflow Setup

`mlflow.statsmodels.autolog` is **disabled**:

- It saved a pickled statsmodels model (plus a full `uv` environment export) for every run, which took about 90% of each run's time. A SARIMAX fit takes 0.02–0.5 s, while a run with autolog took about 5 s.
- The grid fits run in parallel worker processes, where autolog would not work.
- The `AutoStationaryTransformer` tests (ADF, White/OLS) call statsmodels `fit` internally, and autolog would log each of them as a separate run.

Each grid run logs its parameters and metrics explicitly. Only the **best** model is saved as a model artifact, at the end of the notebook (*Save the Best Model to MLflow*).

The tracking server runs in Docker (`docker compose up -d mlflow`, or `task mlflow`) and is reached through `MLFLOW_TRACKING_URI=http://localhost:5000` in `.env`.

```
In [6]: mlflow.statsmodels.autolog(disable=True)

print(f'Tracking URI: {mlflow.get_tracking_uri()}')
print(f'Experiment : {experiment.name} (id {experiment.experiment_id})')
```

```
Tracking URI: http://localhost:5000
Experiment : Brazil Tourism Forecasting ARIMA (id 1)
```

```
In [7]: n_val = len(val_monthly)

# physical cores: SARIMAX fits are CPU-bound, hyper-threads add little
N_JOBS = max(1, (os.cpu_count() or 2) // 2)

def forecast_arrivals(fit, cfg, steps=n_val, index=val_monthly.index):
    """Forecast on the transformed scale and inverse-transform to arrival counts."""
    transformed = np.asarray(fit.forecast(steps=steps, exog=cfg['exog_val']))
    forecast = cfg['transformer'].inverse_transform(pd.Series(transformed, index=index))
    return forecast.to_numpy()

def fit_one(data_key, transform_name, cfg, order_spec):
    """Fit and score one grid point. Runs in a worker process – no MLflow calls."""
    p, d, q, P, D, Q = order_spec
    order = (p, d, q)
    seasonal_order = (P, D, Q, s)
    result = {
        'data_selection': data_key,
        'target_transform': transform_name,
        'run_name': (
            f'SARIMAX({p},{d},{q})({P},{D},{Q})[{s}] ({data_key} | {transform_name})',
        ),
        'order': order,
```

```

        'seasonal_order': seasonal_order,
        'd': d,
        'D': D,
    }
    start = time.perf_counter()
    try:
        with warnings.catch_warnings():
            warnings.simplefilter('ignore')
            fit = SARIMAX(
                cfg['series'],
                exog=cfg['exog_train'],
                order=order,
                seasonal_order=seasonal_order,
                enforce_stationarity=False,
                enforce_invertibility=False,
            ).fit(dispatch=False)

            forecast = forecast_arrivals(fit, cfg)
            if not np.all(np.isfinite(forecast)):
                raise ValueError('non-finite forecast after inverse transform')

            result.update({
                'aicc': fit.aicc,
                'aic': fit.aic,
                'bic': fit.bic,
                **score(val_actual, forecast),
                'converged': True,
                'error': None,
            })
    except Exception as exc:
        result.update({
            'aic': np.inf,
            'bic': np.inf,
            'rmse': np.inf,
            'mae': np.inf,
            'mape': np.inf,
            'r2': -np.inf,
            'mase': np.inf,
            'forecast_bias': np.nan,
            'converged': False,
            'error': str(exc)[:250],
        })
    result['fit_seconds'] = time.perf_counter() - start
    return result

# — phase 1: fit every grid point in parallel —————
tasks = [
    (data_key, transform_name, cfg, order_spec)
    for (data_key, transform_name), cfg in model_configs.items()
    for order_spec in sarimax_grid
]
print(f'Fitting {len(tasks)} models on {N_JOBS} workers...')

grid_start = time.perf_counter()
with parallel_config(backend='loky', inner_max_num_threads=1):

```

```

        results = Parallel(n_jobs=N_JOBS, verbose=5)(
            delayed(fit_one)(*task) for task in tasks
        )
    print(f'Fitting done in {time.perf_counter() - grid_start:,.0f}s')

# — phase 2: Log every result to MLflow from the main process —————
def log_result(client, result):
    cfg = model_configs[(result['data_selection'], result['target_transform'])]
    p, d, q = result['order']
    P, D, Q, _ = result['seasonal_order']
    exog_label = (
        '+' .join(cfg['exog_train'].columns) if cfg['exog_train'] is not None else ''
    )
    params = {
        'p': p,
        'd': d,
        'q': q,
        'P': P,
        'D': D,
        'Q': Q,
        's': s,
        'transform': result['target_transform'],
        'ast_pipeline': cfg['pipeline'],
        'boxcox_lambda': cfg['boxcox_lambda'] or 'none',
        'data_selection': result['data_selection'],
        'exog': exog_label,
        'n_train': len(cfg['series']),
        'n_val': n_val,
        'eval_split': 'validation',
    }
    if result['error']:
        params['error'] = result['error']

    metric_keys = ['aicc', 'aic', 'bic', *METRICS, 'fit_seconds']
    timestamp = int(time.time() * 1000)
    metrics = [
        mlflow.entities.Metric(key, float(result[key]), timestamp, 0)
        for key in metric_keys
        if key in result and np.isfinite(result[key])
    ]

    run = client.create_run(experiment.experiment_id, run_name=result['run_name'])
    client.log_batch(
        run.info.run_id,
        metrics=metrics,
        params=[mlflow.entities.Param(k, str(v)) for k, v in params.items()],
    )
    client.set_terminated(
        run.info.run_id, status='FINISHED' if result['converged'] else 'FAILED'
    )

# each run is 3 HTTP calls; threads overlap the request latency (~3x faster)
N_LOG_THREADS = 8

```

```

client = mlflow.MlflowClient()
log_start = time.perf_counter()
with ThreadPoolExecutor(max_workers=N_LOG_THREADS) as executor:
    list(
        tqdm(
            executor.map(lambda result: log_result(client, result), results),
            total=len(results),
            desc='Logging to MLflow',
        )
    )
print(f'Logging done in {time.perf_counter() - log_start:,.0f}s')

results_df = (
    pd.DataFrame(results).query('converged').sort_values('rmse').reset_index(drop=True)
)
results_df['beats_seasonal_naive'] = results_df['rmse'] < BASELINE_RMSE

print(f'Converged: {len(results_df)} / {len(tasks)}')
print(
    f'{results_df["beats_seasonal_naive"].sum()} models beat the seasonal naive '
    f'(validation RMSE {BASELINE_RMSE:,.0f})'
)
results_df.head(10)

```

Fitting 6048 models on 8 workers...

```

[Parallel(n_jobs=8)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done   2 tasks      | elapsed:    5.1s
[Parallel(n_jobs=8)]: Done  56 tasks      | elapsed:    5.9s
[Parallel(n_jobs=8)]: Done 228 tasks      | elapsed:   11.6s
[Parallel(n_jobs=8)]: Done 392 tasks      | elapsed:   17.3s
[Parallel(n_jobs=8)]: Done 554 tasks      | elapsed:   22.8s
[Parallel(n_jobs=8)]: Done 752 tasks      | elapsed:   32.3s
[Parallel(n_jobs=8)]: Done 1020 tasks     | elapsed:   41.4s
[Parallel(n_jobs=8)]: Done 1560 tasks     | elapsed:   58.7s
[Parallel(n_jobs=8)]: Done 1988 tasks     | elapsed:  1.2min
[Parallel(n_jobs=8)]: Done 2672 tasks     | elapsed:  1.4min
[Parallel(n_jobs=8)]: Done 3428 tasks     | elapsed:  1.7min
[Parallel(n_jobs=8)]: Done 4256 tasks     | elapsed:  2.0min
[Parallel(n_jobs=8)]: Done 5156 tasks     | elapsed:  2.2min
[Parallel(n_jobs=8)]: Done 6048 out of 6048 | elapsed:  2.5min finished

```

Fitting done in 150s

Logging to MLflow: 100% ██████████ | 6048/6048 [16:36<00:00, 6.07it/s]

Logging done in 997s

Converged: 5998 / 6048

2043 models beat the seasonal naive (validation RMSE 255,353)

Out[7]:

	data_selection	target_transform	run_name	order	seasonal_order
0	last_8y_covid_recovery_exog	log	SARIMAX(2,1,2)(1,1,0) [12] (last_8y_covid_recov...	(2, 1, 2)	(1, 1, 0, 12)
1	last_8y_covid_exog	log	SARIMAX(2,1,2)(1,1,0) [12] (last_8y_covid_exog ...	(2, 1, 2)	(1, 1, 0, 12)
2	last_4y_covid_recovery_exog	log	SARIMAX(2,1,0)(0,1,1) [12] (last_4y_covid_recov...	(2, 1, 0)	(0, 1, 1, 12)
3	last_10y_covid_exog	log	SARIMAX(2,1,1)(1,1,0) [12] (last_10y_covid_exog...	(2, 1, 1)	(1, 1, 0, 12)
4	last_8y_covid_exog	log	SARIMAX(1,1,2)(1,1,0) [12] (last_8y_covid_exog ...	(1, 1, 2)	(1, 1, 0, 12)
5	post_covid	log	SARIMAX(0,0,0)(1,0,1) [12] (post_covid log)	(0, 0, 0)	(1, 0, 1, 12)
6	last_8y_covid_recovery_exog	log	SARIMAX(1,1,2)(1,1,0) [12] (last_8y_covid_recov...	(1, 1, 2)	(1, 1, 0, 12)
7	post_covid	log	SARIMAX(0,0,1)(1,0,1) [12] (post_covid log)	(0, 0, 1)	(1, 0, 1, 12)
8	last_8y_covid_recovery_exog	log	SARIMAX(1,1,1)(1,1,0) [12] (last_8y_covid_recov...	(1, 1, 1)	(1, 1, 0, 12)
9	last_8y_covid_exog	log	SARIMAX(2,1,1)(1,1,0) [12] (last_8y_covid_exog ...	(2, 1, 1)	(1, 1, 0, 12)

Results

```
In [8]: palette = {
    'last_14y_covid_exog': 'midnightblue',
    'last_12y_covid_exog': 'steelblue',
    'last_10y_covid_exog': 'slateblue',
    'last_8y_covid_exog': 'mediumpurple',
    'last_6y_covid_exog': 'orchid',
    'last_4y_covid_exog': 'palevioletred',
    'last_14y_covid_recovery_exog': 'darkslategray',
    'last_12y_covid_recovery_exog': 'teal',
    'last_10y_covid_recovery_exog': 'cadetblue',
    'last_8y_covid_recovery_exog': 'mediumaquamarine',
    'last_6y_covid_recovery_exog': 'darkturquoise',
    'last_4y_covid_recovery_exog': 'lightseagreen',
```

```

        'post_covid': 'seagreen',
        'post_covid_recovery_exog': 'darkolivegreen',
    }

best_per_selection = (
    results_df
    .sort_values('rmse')
    .groupby('data_selection', sort=False)
    .first()
    .reset_index()
    .sort_values('rmse')
    .reset_index(drop=True)
)
best_per_selection['label'] = best_per_selection['run_name'].str.replace(
    'SARIMAX', '', regex=False
)
bar_colors = [palette[ds] for ds in best_per_selection['data_selection'][::-1]]

fig, axes = plt.subplots(2, 3, figsize=(22, 10), sharey=True)
fig.suptitle(
    'Best SARIMAX Model per Data Selection (Validation Set) - '
    'dashed line = seasonal naive',
    fontsize=14,
    fontweight='bold',
)

for ax, col in zip(axes.flat, METRICS):
    title = METRIC_LABELS[col]
    ax.barh(
        best_per_selection['label'][::-1],
        best_per_selection[col][::-1],
        color=bar_colors,
        edgecolor='white',
    )
    ax.axvline(
        float(baselines.loc['seasonal_naive', col]),
        color='grey',
        linestyle='--',
    )
    if col == 'forecast_bias':
        ax.axvline(0, color='black', linewidth=0.8)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel(title)
    sns.despine(ax=ax)

plt.tight_layout()
plt.show()

display(
    best_per_selection[
        [
            'data_selection',
            'target_transform',
            'run_name',
            *METRICS,
            'beats_seasonal_naive',

```



	data_selection	target_transform	run_name	rmse	
0	last_8y_covid_recovery_exog	log	SARIMAX(2,1,2)(1,1,0) [12] (last_8y_covid_recov...	60028.041890	46428.
1	last_8y_covid_exog	log	SARIMAX(2,1,2)(1,1,0) [12] (last_8y_covid_exog ...	60476.468022	51478.
2	last_4y_covid_recovery_exog	log	SARIMAX(2,1,0)(0,1,1) [12] (last_4y_covid_recov...	62627.959257	56162.
3	last_10y_covid_exog	log	SARIMAX(2,1,1)(1,1,0) [12] (last_10y_covid_exog...	72396.147940	61144.
4	post_covid	log	SARIMAX(0,0,0)(1,0,1) [12] (post_covid log)	76282.690864	47690.
5	last_6y_covid_recovery_exog	log	SARIMAX(1,1,1)(1,1,0) [12] (last_6y_covid_recov...	102886.917506	72359.
6	post_covid_recovery_exog	log	SARIMAX(2,1,2)(0,1,1) [12] (post_covid_recovery...	104373.073653	63881.
7	last_10y_covid_recovery_exog	log	SARIMAX(1,1,1)(1,1,0) [12] (last_10y_covid_reco...	107965.267245	77839.
8	last_12y_covid_exog	log	SARIMAX(2,1,1)(1,1,0) [12] (last_12y_covid_exog...	108154.733430	78425.
9	last_14y_covid_exog	log	SARIMAX(2,1,1)(1,1,0) [12] (last_14y_covid_exog...	108584.637581	78839.
10	last_12y_covid_recovery_exog	log	SARIMAX(0,1,2)(1,1,0) [12] (last_12y_covid_reco...	108823.199479	79037.
11	last_14y_covid_recovery_exog	log	SARIMAX(2,1,0)(1,1,0) [12] (last_14y_covid_reco...	109766.144874	79981.
12	last_6y_covid_exog	log	SARIMAX(0,1,2)(1,1,0) [12] (last_6y_covid_exog ...	113701.702213	82896.
13	last_4y_covid_exog	log	SARIMAX(0,0,0)(1,0,1) [12] (last_4y_covid_exog ...	116896.169714	95472.

Effect of Target Transformation and Differencing

Best validation RMSE for each target transformation, within each data selection and for each (d, D) combination. Every cell is the best model of its group across all remaining orders. These tables answer whether a transformation helps, and whether it replaces or still needs differencing.

```
In [9]: best_by_selection_transform = results_df.pivot_table(
        index='data_selection', columns='target_transform', values='rmse', aggfunc='min'
    ).reindex(columns=TARGET_TRANSFORMS)
    best_by_selection_transform['best'] = best_by_selection_transform.idxmin(axis=1)

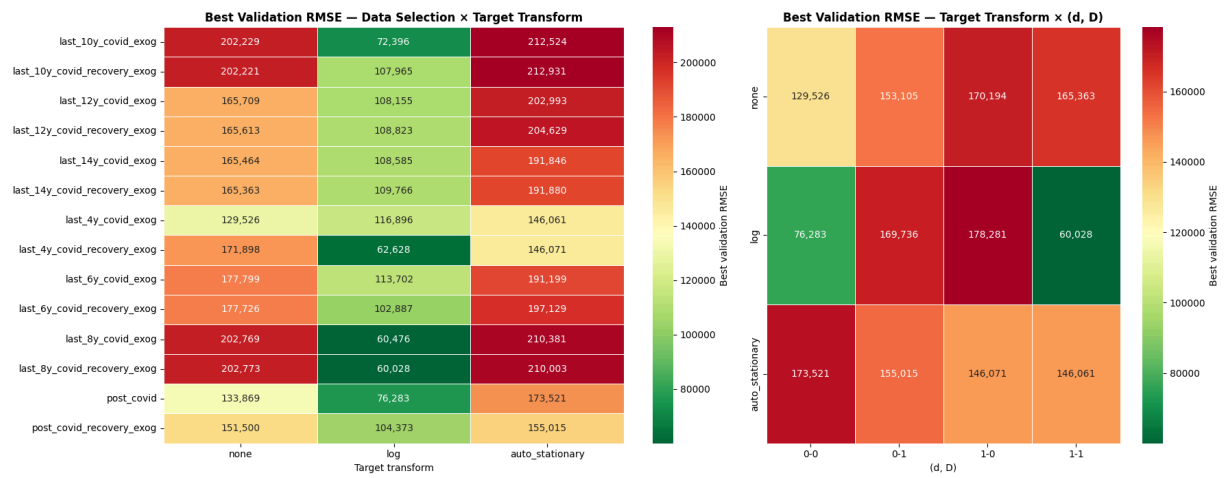
    best_by_transform_diff = results_df.pivot_table(
        index='target_transform', columns=['d', 'D'], values='rmse', aggfunc='min'
    ).reindex(TARGET_TRANSFORMS)

    fig, axes = plt.subplots(1, 2, figsize=(18, 7), gridspec_kw={'width_ratios': [1.3,
    sns.heatmap(
        best_by_selection_transform[TARGET_TRANSFORMS],
        annot=True,
        fmt=',.0f',
        cmap='RdYlGn_r',
        linewidths=0.4,
        ax=axes[0],
        cbar_kws={'label': 'Best validation RMSE'},
    )
    axes[0].set_title(
        'Best Validation RMSE – Data Selection × Target Transform', fontweight='bold'
    )
    axes[0].set_xlabel('Target transform')
    axes[0].set_ylabel('')

    sns.heatmap(
        best_by_transform_diff,
        annot=True,
        fmt=',.0f',
        cmap='RdYlGn_r',
        linewidths=0.4,
        ax=axes[1],
        cbar_kws={'label': 'Best validation RMSE'},
    )
    axes[1].set_title('Best Validation RMSE – Target Transform × (d, D)', fontweight='b
    axes[1].set_xlabel('(d, D)')
    axes[1].set_ylabel('')

    plt.tight_layout()
    plt.show()

    display(best_by_selection_transform.round(0))
    display(best_by_transform_diff.round(0))
```



target_transform		none	log	auto_stationary	best
data_selection					
last_10y_covid_exog		202229.0	72396.0	212524.0	log
last_10y_covid_recovery_exog		202221.0	107965.0	212931.0	log
last_12y_covid_exog		165709.0	108155.0	202993.0	log
last_12y_covid_recovery_exog		165613.0	108823.0	204629.0	log
last_14y_covid_exog		165464.0	108585.0	191846.0	log
last_14y_covid_recovery_exog		165363.0	109766.0	191880.0	log
last_4y_covid_exog		129526.0	116896.0	146061.0	log
last_4y_covid_recovery_exog		171898.0	62628.0	146071.0	log
last_6y_covid_exog		177799.0	113702.0	191199.0	log
last_6y_covid_recovery_exog		177726.0	102887.0	197129.0	log
last_8y_covid_exog		202769.0	60476.0	210381.0	log
last_8y_covid_recovery_exog		202773.0	60028.0	210003.0	log
post_covid		133869.0	76283.0	173521.0	log
post_covid_recovery_exog		151500.0	104373.0	155015.0	log

d		0		1	
D		0	1	0	1
target_transform					
none		129526.0	153105.0	170194.0	165363.0
log		76283.0	169736.0	178281.0	60028.0
auto_stationary		173521.0	155015.0	146071.0	146061.0

Best Model Diagnostics

```
In [10]: top10 = results_df.head(10).copy()

display(
    top10[
        [
            'data_selection',
            'run_name',
            'aicc',
            'aic',
            'bic',
            *METRICS,
        ]
    ]
)
```

	data_selection	run_name	aicc	aic	bic	
0	last_8y_covid_recovery_exog	SARIMAX(2,1,2)(1,1,0) [12] (last_8y_covid_recov...	207.363257	205.002601	222.990563	60026
1	last_8y_covid_exog	SARIMAX(2,1,2)(1,1,0) [12] (last_8y_covid_exog ...	204.797213	202.990762	218.730228	60476
2	last_4y_covid_recovery_exog	SARIMAX(2,1,0)(0,1,1) [12] (last_4y_covid_recov...	-0.550151	-5.800151	1.012814	62627
3	last_10y_covid_exog	SARIMAX(2,1,1)(1,1,0) [12] (last_10y_covid_exog...	241.031624	240.066107	255.325876	72396
4	last_8y_covid_exog	SARIMAX(1,1,2)(1,1,0) [12] (last_8y_covid_exog ...	206.293454	204.980954	218.557033	74974
5	post_covid	SARIMAX(0,0,0)(1,0,1) [12] (post_covid log)	2.758611	1.158611	3.991928	76282
6	last_8y_covid_recovery_exog	SARIMAX(1,1,2)(1,1,0) [12] (last_8y_covid_recov...	208.756027	206.978249	222.817008	76626
7	post_covid	SARIMAX(0,0,1)(1,0,1) [12] (post_covid log)	-2.247758	-5.324682	-1.763195	81767
8	last_8y_covid_recovery_exog	SARIMAX(1,1,1)(1,1,0) [12] (last_8y_covid_recov...	204.276637	202.964137	216.540216	85617
9	last_8y_covid_exog	SARIMAX(2,1,1)(1,1,0) [12] (last_8y_covid_exog ...	203.039113	201.705780	215.196751	88123

```
In [11]: def fit_model(row, configs):
          cfg = configs[(row['data_selection'], row['target_transform'])]
          return SARIMAX(
              cfg['series'],
              exog=cfg['exog_train'],
              order=row['order'],
              seasonal_order=row['seasonal_order'],
              enforce_stationarity=False,
              enforce_invertibility=False,
          ).fit(dispatch=False)

          top10_fits = [fit_model(row, model_configs) for _, row in top10.iterrows()]

          for i, (_, row) in enumerate(top10.iterrows()):
              print(f'#{i + 1}>2} [{row["data_selection"]} {row["run_name"]}')
```

```

# 1 [last_8y_covid_recovery_exog] SARIMAX(2,1,2)(1,1,0)[12] (last_8y_covid_recovery_exog | log)
# 2 [last_8y_covid_exog] SARIMAX(2,1,2)(1,1,0)[12] (last_8y_covid_exog | log)
# 3 [last_4y_covid_recovery_exog] SARIMAX(2,1,0)(0,1,1)[12] (last_4y_covid_recovery_exog | log)
# 4 [last_10y_covid_exog] SARIMAX(2,1,1)(1,1,0)[12] (last_10y_covid_exog | log)
# 5 [last_8y_covid_exog] SARIMAX(1,1,2)(1,1,0)[12] (last_8y_covid_exog | log)
# 6 [post_covid] SARIMAX(0,0,0)(1,0,1)[12] (post_covid | log)
# 7 [last_8y_covid_recovery_exog] SARIMAX(1,1,2)(1,1,0)[12] (last_8y_covid_recovery_exog | log)
# 8 [post_covid] SARIMAX(0,0,1)(1,0,1)[12] (post_covid | log)
# 9 [last_8y_covid_recovery_exog] SARIMAX(1,1,1)(1,1,0)[12] (last_8y_covid_recovery_exog | log)
#10 [last_8y_covid_exog] SARIMAX(2,1,1)(1,1,0)[12] (last_8y_covid_exog | log)

```

Residual Diagnostics

```

In [12]: fig, axes = plt.subplots(10, 3, figsize=(16, 40))
fig.suptitle(
    'Residual Diagnostics - Top 10 Models', fontsize=14, fontweight='bold', y=1.001
)

x_qq = np.linspace(-4, 4, 200)

for i, (_, row), fit in enumerate(zip(top10.iterrows(), top10_fits)):
    std_resid = fit.filter_results.standardized_forecasts_error[0]
    std_resid = std_resid[~np.isnan(std_resid)]

    row_label = f'#{i + 1} {row["run_name"].replace("SARIMAX", "")}'

    # Standardized residuals over time
    axes[i, 0].plot(std_resid, linewidth=1.2, color=palette[row['data_selection']])
    axes[i, 0].axhline(0, color='black', linestyle='--', alpha=0.5)
    axes[i, 0].set_title(f'{row_label}\nStandardized Residuals', fontsize=8)
    axes[i, 0].set_xlabel('Observation', fontsize=8)
    sns.despine(ax=axes[i, 0])

    # Histogram + KDE + N(0,1)
    axes[i, 1].hist(
        std_resid,
        bins='auto',
        density=True,
        alpha=0.5,
        color=palette[row['data_selection']],
        label='Hist',
    )
    axes[i, 1].plot(x_qq, stats.norm.pdf(x_qq, 0, 1), 'r-', label='N(0,1)')
    kde = stats.gaussian_kde(std_resid)
    axes[i, 1].plot(x_qq, kde(x_qq), 'g-', label='KDE')
    axes[i, 1].set_title('Histogram plus Estimated Density', fontsize=8)
    axes[i, 1].legend(fontsize=7)
    sns.despine(ax=axes[i, 1])

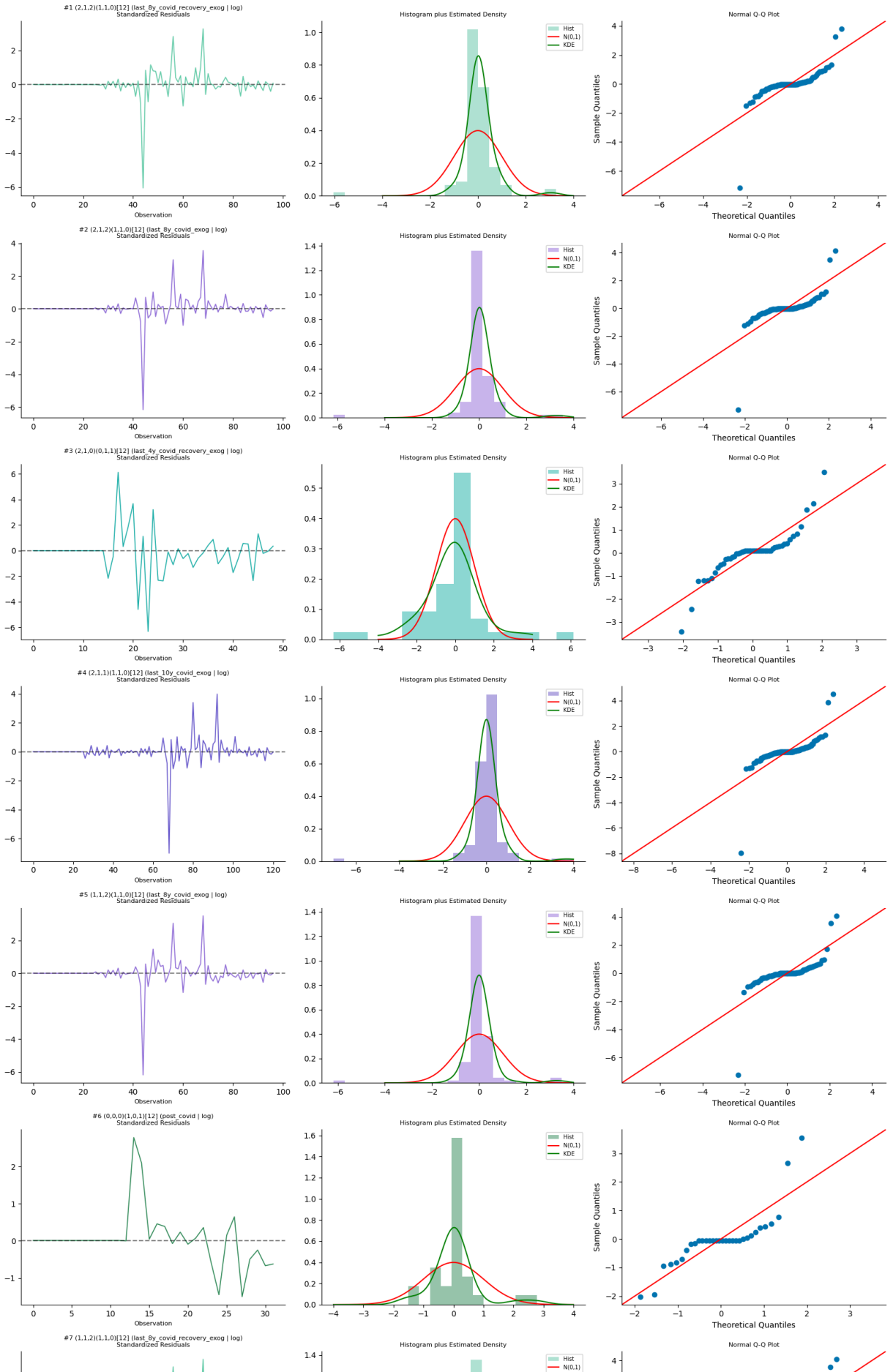
    # Normal Q-Q plot
    sm.qqplot(std_resid, line='45', fit=True, ax=axes[i, 2])

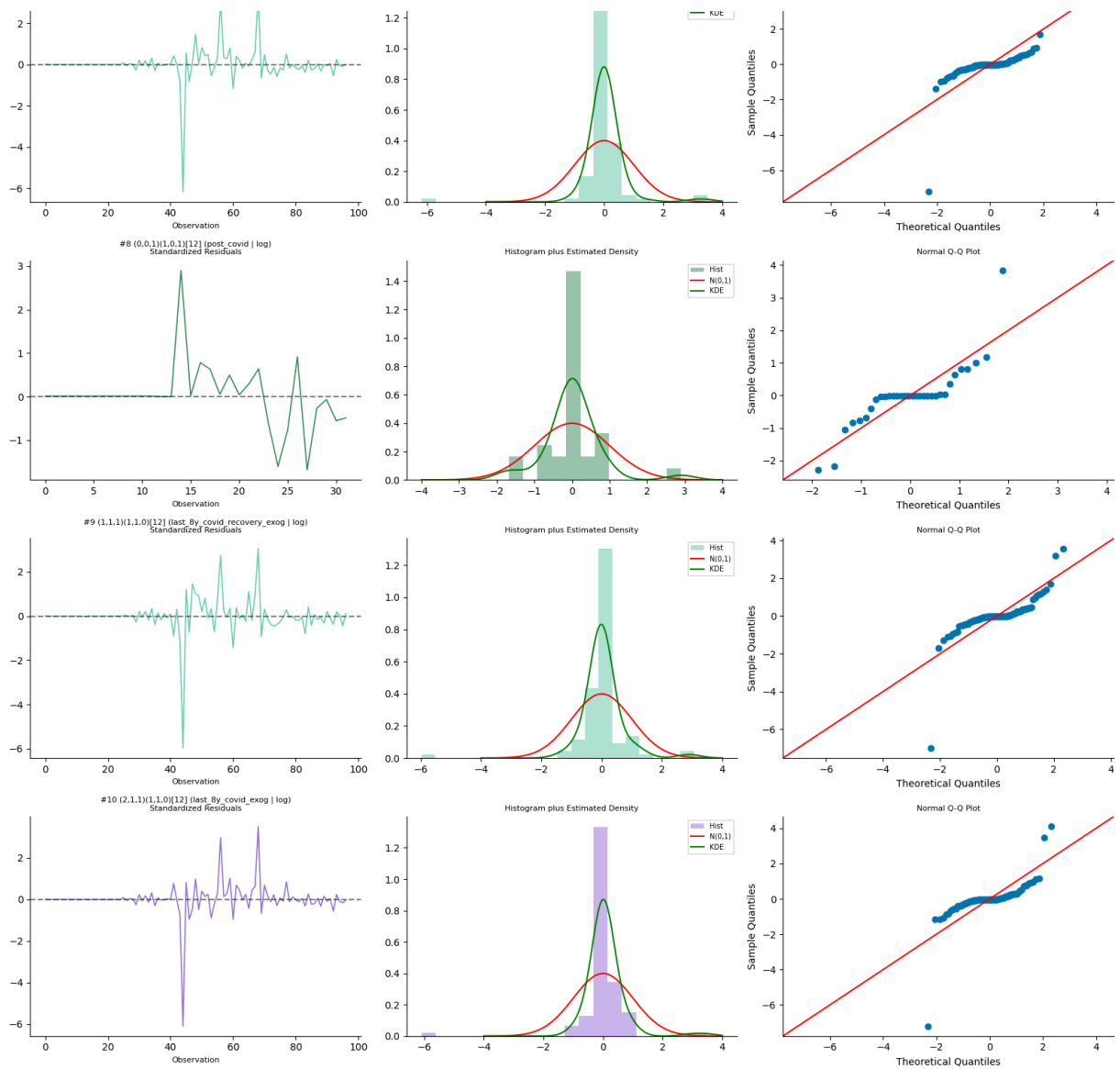
```

```
axes[i, 2].set_title('Normal Q-Q Plot', fontsize=8)
sns.despine(ax=axes[i, 2])

plt.tight_layout()
plt.show()
```

Residual Diagnostics — Top 10 Models





Final Result

```
In [13]: fig, axes = plt.subplots(5, 2, figsize=(14, 22))
fig.suptitle(
    f'Top 10 Models – Validation Set Forecasts '
    f'({val_monthly.index[0]:%b %Y}-{val_monthly.index[-1]:%b %Y})',
    fontsize=15,
    fontweight='bold',
)

for i, (ax, (_, row), fit) in enumerate(zip(axes.flat, top10.iterrows(), top10_fits)):
    cfg = model_configs[(row['data_selection'], row['target_transform'])]
    forecast = forecast_arrivals(fit, cfg)
    train_plot = cfg['raw_series'].iloc[-24:]

    ax.plot(
        train_plot.index,
        train_plot.values,
        label='Train (last 24 m)',
        linewidth=1.5,
```

```

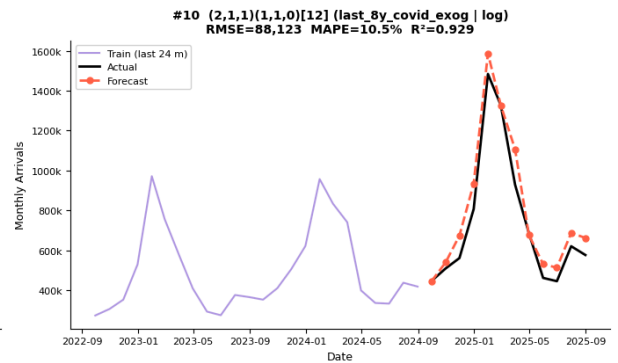
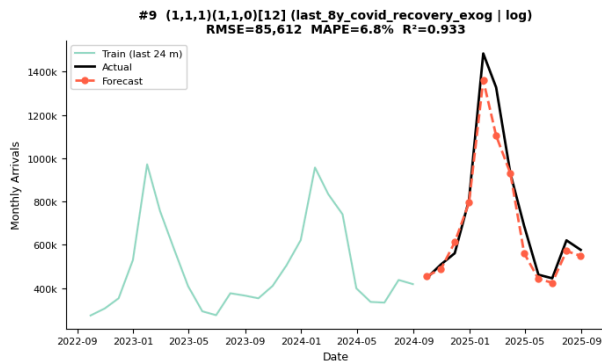
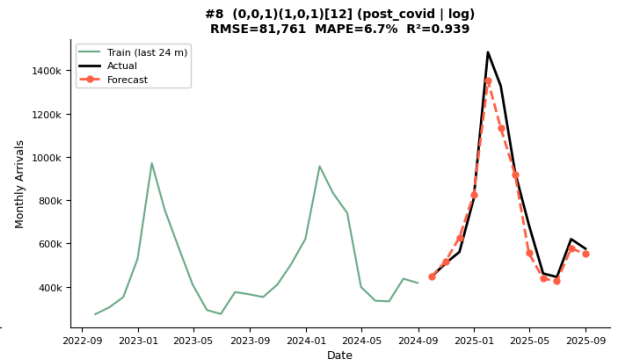
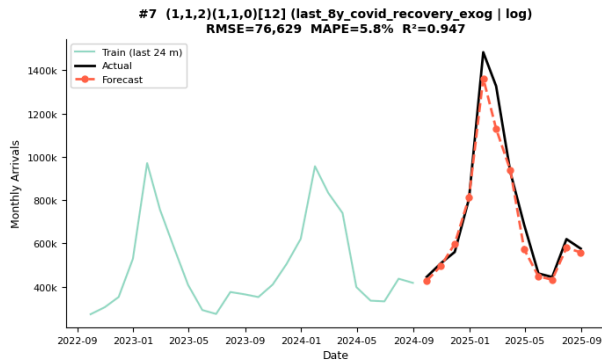
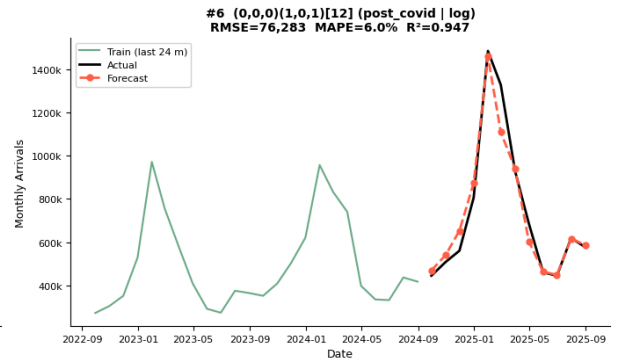
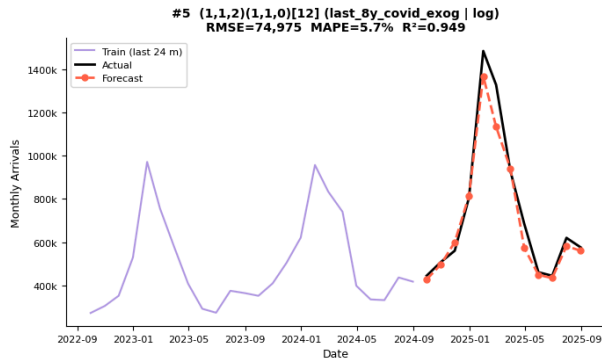
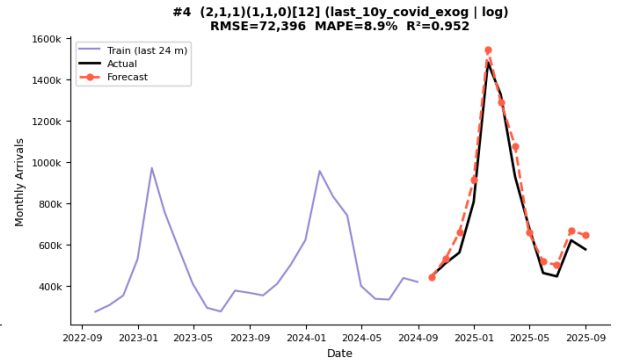
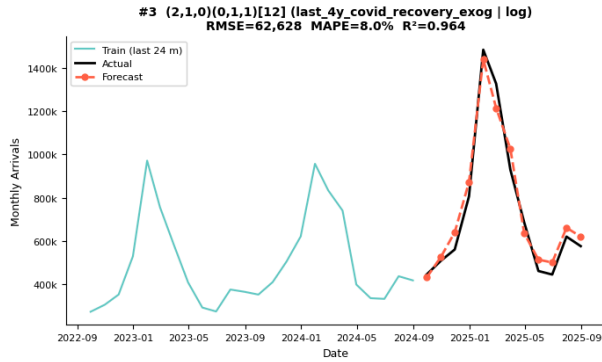
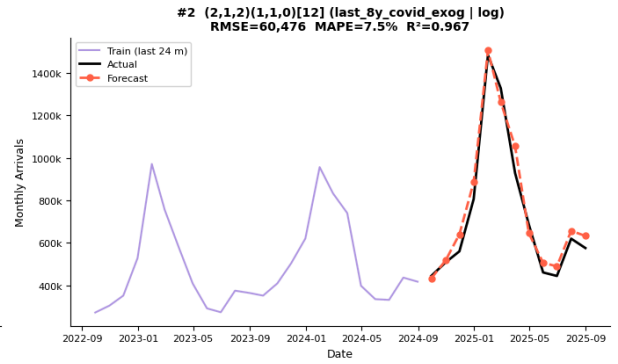
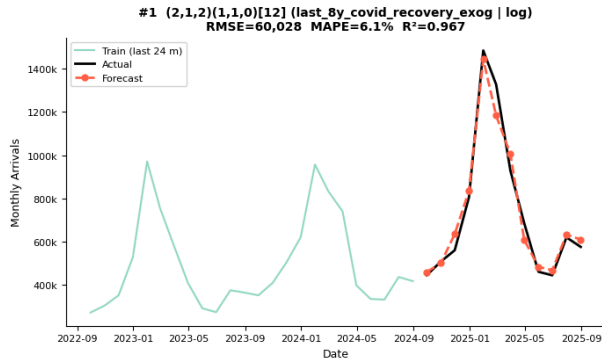
        color=palette[row['data_selection']],
        alpha=0.7,
    )
    ax.plot(
        val_monthly.index,
        val_monthly['arrival_count'],
        label='Actual',
        linewidth=2,
        color='black',
    )
    ax.plot(
        val_monthly.index,
        forecast,
        label='Forecast',
        linewidth=2,
        linestyle='--',
        color='tomato',
        marker='o',
        markersize=5,
    )

    ax.set_title(
        f'#{i + 1} {row["run_name"].replace("SARIMAX", "")}\n'
        f'RMSE={row["rmse"]:.0f} MAPE={row["mape"]:.1f}% R²={row["r2"]:.3f}',
        fontsize=10,
        fontweight='bold',
    )
    ax.set_xlabel('Date', fontsize=9)
    ax.set_ylabel('Monthly Arrivals', fontsize=9)
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _ : f'{int(x / 1000)}k')
    ax.tick_params(axis='both', labelsize=8)
    ax.legend(fontsize=8)
    sns.despine(ax=ax)

plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()

```

Top 10 Models — Validation Set Forecasts (Sep 2024-Aug 2025)



Save the Best Model to MLflow

The best model (rank 1 by validation RMSE, across all orders, target transformations and data selections) is saved to MLflow as a single deployable artifact. It is trained on the training set only; the final notebook refits the selected configuration and evaluates it once on the test set.

- **What is saved.** An `mlflow.pyfunc` model that wraps the fitted target transformer (`none` , `log` or `AutoStationaryTransformer`) together with the fitted SARIMAX results. `predict` forecasts on the transformed scale and applies `inverse_transform` internally, so the model returns **arrival counts** directly.
- **Self-contained.** The book's `src` package is bundled with `code_paths` , so an `auto_stationary` model loads on a machine without the sibling repository. Its import-time dependencies (`plotly` , `pymannkendall`) are added to the model's requirements.
- **Registry.** The model is registered as `brazil_tourism_sarimax_baseline` and the new version gets the alias `champion` . Later notebooks (XGBoost, PatchTST) can load the baseline with `models:/brazil_tourism_sarimax_baseline@champion` .
- **Input contract.** A DataFrame with a `date` column holding the consecutive month-ends right after the end of training (the first validation month onwards), plus the exogenous columns the model was trained with (if any). The output has the columns `date` and `arrival_count_forecast` .

The saved model is reloaded from the registry at the end, and its forecast is checked against the in-memory model.

```
In [14]: from mlflow.models import infer_signature # noqa: E402

REGISTERED_MODEL_NAME = 'brazil_tourism_sarimax_baseline'
MODEL_ALIAS = 'champion'

class TransformedSARIMAXModel(mlflow.pyfunc.PythonModel):
    """SARIMAX fitted on a transformed target; predicts arrival counts.

    `model_input` must hold the consecutive month-ends right after the end of train
    in a `date` column, plus the exogenous columns used at training time.
    """

    def __init__(self, transformer, sarimax_results, exog_columns):
        self.transformer = transformer
        self.sarimax_results = sarimax_results
        self.exog_columns = exog_columns

    def predict(self, context, model_input, params=None):
        dates = pd.DatetimeIndex(pd.to_datetime(model_input['date']))
        exog = (
            model_input[self.exog_columns].set_index(dates)
```

```

        if self.exog_columns
        else None
    )
    transformed = self.sarimax_results.forecast(steps=len(dates), exog=exog)
    forecast = self.transformer.inverse_transform(
        pd.Series(np.asarray(transformed), index=dates)
    )
    return pd.DataFrame({
        'date': dates,
        'arrival_count_forecast': forecast.to_numpy(),
    })

best_row = results_df.iloc[0]
best_cfg = model_configs[(best_row['data_selection'], best_row['target_transform'])]
best_fit = top10_fits[0]
exog_columns = (
    list(best_cfg['exog_train'].columns) if best_cfg['exog_train'] is not None else
)

python_model = TransformedSARIMAXModel(best_cfg['transformer'], best_fit, exog_colu
model_input = val_monthly.reset_index()[['date', *exog_columns]]
best_prediction = python_model.predict(None, model_input)
signature = infer_signature(model_input, best_prediction)

print(f'Best model : {best_row["run_name"]}')
print(f'Transform : {best_cfg["target_transform"]} ({best_cfg["pipeline"]})')
print(
    f'Val RMSE : {best_row["rmse"]:.0f} | MAPE: {best_row["mape"]:.2f}% | '
    f'R²: {best_row["r2"]:.3f} | MASE: {best_row["mase"]:.3f} | '
    f'Bias: {best_row["forecast_bias"]:.2f}%'
)

```

Best model : SARIMAX(2,1,2)(1,1,0)[12] (last_8y_covid_recovery_exog | log)

Transform : log (log)

Val RMSE : 60,028 | MAPE: 6.08% | R²: 0.967 | MASE: 0.560 | Bias: -0.22%

c:\git_reps\brazil_tourism_forecasting_benchmark\.venv\Lib\site-packages\mlflow\pyfun
c\utils\data_validation.py:187: UserWarning: Add type hints to the `predict` method
to enable data validation and automatic signature inference during model logging. Ch
eck https://mlflow.org/docs/latest/model/python_model.html#type-hint-usage-in-python
model for more details.

color_warning(

In [15]: with mlflow.start_run(run_name=f'BEST {best_row["run_name"]}') as best_run:

```

    p, d_, q = best_row['order']
    P, D_, Q, s_ = best_row['seasonal_order']
    mlflow.log_params({
        'p': p,
        'd': d_,
        'q': q,
        'P': P,
        'D': D_,
        'Q': Q,
        's': s_,
        'transform': best_cfg['target_transform'],
        'ast_pipeline': best_cfg['pipeline'],
    })

```

```

        'boxcox_lambda': best_cfg['boxcox_lambda'] or 'none',
        'data_selection': best_row['data_selection'],
        'exog': '+' . join(exog_columns) if exog_columns else 'none',
        'n_train': len(best_cfg['series']),
        'n_val': n_val,
        'eval_split': 'validation',
        'selection_metric': 'val_rmse',
    })
    mlflow.log_metrics({key: float(best_row[key]) for key in [*METRICS, 'aic', 'bic']})
    mlflow.set_tags({
        'role': 'baseline',
        'model_family': 'SARIMAX',
        'target_transform': best_cfg['target_transform'],
    })
    model_info = mlflow.pyfunc.log_model(
        name='model',
        python_model=python_model,
        code_paths=[os.path.join(MTSF_REPO_PATH, 'src')],
        signature=signature,
        input_example=model_input,
        registered_model_name=REGISTERED_MODEL_NAME,
        extra_pip_requirements=['plotly', 'pymannkendall'],
    )

    mlflow.MlflowClient().set_registered_model_alias(
        REGISTERED_MODEL_NAME, MODEL_ALIAS, model_info.registered_model_version
    )

    print(f'Run ID           : {best_run.info.run_id}')
    print(f'Model URI          : {model_info.model_uri}')
    print(
        f'Registered         : {REGISTERED_MODEL_NAME} '
        f'v{model_info.registered_model_version} (@{MODEL_ALIAS})'
    )

```

Downloading artifacts: 0%| | 0/39 [00:00<?, ?it/s]

Successfully registered model 'brazil_tourism_sarimax_baseline'.
Created version '1' of model 'brazil_tourism_sarimax_baseline'.

```

Run ID           : e16385f3fb5445409f7563f9b2201d51
Model URI        : models:/m-ccc28384eaa74321aa6316fc0860243e
Registered       : brazil_tourism_sarimax_baseline v1 (@champion)

```

```

In [16]: loaded_model = mlflow.pyfunc.load_model(
        f'models://{REGISTERED_MODEL_NAME}@{MODEL_ALIAS}'
    )
    reloaded_prediction = loaded_model.predict(model_input)

    max_abs_diff = np.max(
        np.abs(
            reloaded_prediction['arrival_count_forecast'].to_numpy()
            - best_prediction['arrival_count_forecast'].to_numpy()
        )
    )
    reloaded_rmse = score(val_actual, reloaded_prediction['arrival_count_forecast'])['rmse']

    print(f'Max |reloaded - in-memory| : {max_abs_diff:.6f}')

```

```
print(f'Reloaded validation RMSE : {reloaded_rmse:,.0f}')
reloaded_prediction.assign(actual=val_actual.to_numpy())
```

Downloading artifacts: 0%| | 0/39 [00:00<?, ?it/s]
Max |reloaded - in-memory| : 0.000000
Reloaded validation RMSE : 60,028

Out[16]:

	date	arrival_count_forecast	actual
0	2024-09-30	4.599637e+05	445389.0
1	2024-10-31	5.013218e+05	508738.0
2	2024-11-30	6.361259e+05	560732.0
3	2024-12-31	8.356359e+05	806478.0
4	2025-01-31	1.443017e+06	1483669.0
5	2025-02-28	1.183246e+06	1326884.0
6	2025-03-31	1.005560e+06	929096.0
7	2025-04-30	6.093168e+05	686239.0
8	2025-05-31	4.842182e+05	461341.0
9	2025-06-30	4.681009e+05	444882.0
10	2025-07-31	6.328063e+05	620143.0
11	2025-08-31	6.102174e+05	576049.0